



Khoa Công nghệ thông tin - Học viện Kỹ thuật quân sự
Trụ sở: Tầng 19, nhà S1, 236 Hoàng Quốc Việt, Cầu Giấy, Hà Nội
Tel: (+84) 069515329 - Email: fit@mta.edu.vn
Website: <https://fit.mta.edu.vn>

<VOCABOOST – HỆ THỐNG HỌC TỪ VỰNG TOEIC THÔNG MINH>

THIẾT KẾ CHI TIẾT

Mã dự án	<2026_VocaBoost_MTA>
Phiên bản	V1.0.0
Ngày	20/06/2026

<HÀ NỘI, 6-2026>

NỘI DUNG SỬA ĐỔI

*M- Mới S – Sửa X - Xóa

[illegible]

TRANG KÝ

NGƯỜI LẬP: Nguyễn Hoàng Phúc _____ <Ngày> _____
<Vị trí: Backend, Architecture>

NGƯỜI KIỂM TRA: Lại Đức Tùng_____ <Ngày> _____
<Vị trí: Team Lead>

NGƯỜI PHÊ DUYỆT: Trần Tuấn Dũng _____ <Ngày> _____
<Vị trí: Frontend>

MỤC LỤC

1. GIỚI THIỆU	6
1.1. Mục đích	6
1.2. Phạm vi.....	6
1.3. Vai trò và trách nhiệm.....	6
1.4. Tài liệu tham khảo	6
1.5. Từ và thuật ngữ	6
2. TỔNG QUAN HỆ THỐNG	8
2.1. Tổng quan.....	8
2.2. Các yêu cầu chức năng.....	8
3. KIẾN TRÚC HỆ THỐNG	9
3.1. Mô hình kiến trúc	9
4. THIẾT KẾ LỚP.....	15
4.1. Class Diagram	15
<i>AuthController</i>	15
<i>LearningPathService</i>	17
<i>AdminService</i>	19
<i>UserService</i>	22
<i>AdminController</i>	24
<i>Word</i>	25
<i>UserService</i>	26
<i>UserWordProgress (SRS Engine)</i>	27
5. THIẾT KẾ DỮ LIỆU	29
6. THIẾT KẾ GIAO DIỆN NGƯỜI DÙNG.....	30
6.1. Trang Dashboard (User) & Landing Page	30
6.2. Popup thông tin từ vựng	31
6.3. Giao diện học từ vựng Flashcard với thẻ SRS và 4 nút đánh giá (Again/Hard/Good/Easy)	32
6.4. Giao diện Mini Test tổng hợp các câu hỏi trắc nghiệm từ vựng	32
6.5. Dialog danh sách từ vựng	33
6.6. Trang Quản trị (Admin Dashboard & CRUD)	33

1. GIỚI THIỆU

1.1. Mục đích

Tài liệu này cung cấp thiết kế chi tiết về kiến trúc, dữ liệu cũng như các thành phần của hệ thống học từ vựng TOEIC thông minh VocaBoost.

Tài liệu này được sử dụng để:

Nhân viên kiểm thử: Lập kịch bản test và số liệu kiểm thử.

Thành viên đội phát triển: Xây dựng các phân hệ nghiệp vụ chức năng.

Nhân viên thiết kế UI/UX: Tham khảo để thiết kế giao diện.

Giảng viên hướng dẫn: Xác nhận tính đúng đắn của thiết kế.

1.2. Phạm vi

Tài liệu này phục vụ cho việc phát triển, bảo trì dự án VocaBoost.

1.3. Vai trò và trách nhiệm

Tài liệu này miêu tả chi tiết các quy trình phát triển các module và sự liên hệ giữa các module trong hệ thống VocaBoost.

Miêu tả chi tiết sự ràng buộc giữa các bảng trong CSDL, vai trò và mục đích các bảng được sử dụng để phát triển hệ thống VocaBoost.

Miêu tả quy trình thực hiện của hệ thống.

1.4. Tài liệu tham khảo

Liệt kê tất cả các tài liệu tham khảo như: các tài liệu khác của hệ thống, hoặc các bài báo kỹ thuật, ...

Loại tài liệu	Tiêu đề

1.5. Từ và thuật ngữ

Từ/thuật ngữ	Ý nghĩa	Diễn giải
<i>API</i>	<i>Application Programming Interface</i>	<i>Giao diện Lập trình Ứng dụng</i>

2. TỔNG QUAN HỆ THỐNG

VocaBoost là hệ thống học từ vựng TOEIC trực tuyến với hơn 720 từ vựng theo 4 cấp độ (300-500-700-900). Người học tiếp nhận và ôn tập từ vựng thông qua phương pháp SRS cùng cơ chế gamification (XP, Level, Streak, Badge).

Hệ thống sử dụng kiến trúc Frontend-Backend-Database với Next.js 16, React 19, Express.js 5 và SQL Server. Authentication dùng JWT kết hợp Bcrypt, phân quyền dựa trên Role-Based Access Control (RBAC) với 3 vai trò: Admin, Learner, ContentCreator.

Hệ thống VocaBoost cung cấp hơn 720 từ vựng TOEIC chia làm 4 cấp độ (300-500-700-900), học theo phương pháp Spaced Repetition System (SRS) với 4 mức đánh giá: Again (lại), Hard (khó), Good (tốt), Easy (dễ). Cơ chế gamification với XP, Level, Streak, Badge khuyến khích học tập hàng ngày.

Hệ thống hỗ trợ ba vai trò người dùng: Người học (Learner) học flashcard SRS, làm mini test, notebook từ vựng. Quản trị viên (Admin) quản lý Words, Questions, Topics, MiniTests, Students, Content Review. Biên tập viên (Content Creator) tạo nội dung và gửi duyệt.

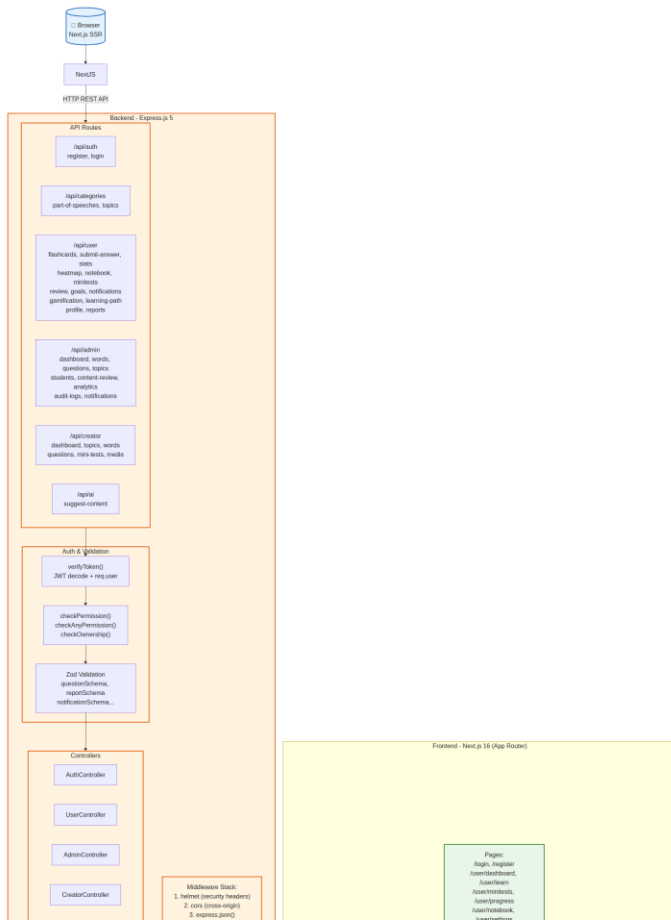
- +) Ảnh hưởng đến chất lượng dịch vụ học tập tại học tập của VocaBoost
- +) + Người học có thể luyện tập từ vựng mới và ôn tập từ đã học theo lịch trình SRS thông minh
- +) + Quản trị viên có thể quản lý từ vựng, câu hỏi, mini test, và duyệt nội dung từ biên tập viên
- + Biên tập viên có thể tạo nội dung từ vựng, câu hỏi, mini test và gửi cho admin duyệt (Draft > PendingReview > Published/Rejected)
- +) Dashboard admin cung cấp thống kê tổng quan: số lượng user, words, questions, mini tests, lượt làm bài, accuracy, và biểu đồ phân tích chi tiết.

2.1. Tổng quan

2.2. Các yêu cầu chức năng

3. KIẾN TRÚC HỆ THỐNG

3.1. Mô hình kiến trúc



Hình 5: Sơ đồ các thành phần của phần mềm

- **Client (Frontend)**

Giao diện người dùng được xây dựng bằng Next.js 16 (App Router), React 19, TypeScript và Tailwind CSS 4. Giao tiếp với backend thông qua REST API (Axios).

- Backend

Thành phần logic xử lý nghiệp vụ của hệ thống (Express.js 5), bao gồm:

- **API Routes (Express Router)**

Các route API tiếp nhận yêu cầu từ Client, xác thực JWT và điều hướng đến Controller tương ứng. Xác thực người dùng dựa trên JWT với phân quyền động.

- **Auth Controller & Auth Service**

Auth Controller & Auth Service xử lý đăng ký và đăng nhập người dùng. AuthService.register() tạo user mới, hash password bằng Bcrypt. AuthService.login() xác thực thông tin và trả về JWT token chứa userId, email, role, permissions.

- **AuthController**

Thành phần Controller xử lý xác thực người dùng. AuthController cung cấp endpoint POST /api/auth/register (đăng ký với fullName, email, password) và POST /api/auth/login (đăng nhập, trả về JWT token). Sử dụng AuthService để hash password bằng Bcrypt, tạo JWT token chứa userId, email, role, permissions với thời hạn 1 ngày. Sau đăng nhập thành công, tự động awardDailyLogin XP cho người học (Learner).

- **AdminController & AdminService**

Admin Controller & Admin Service xử lý logic quản trị: CRUD từ vựng (Words), câu hỏi (Questions), chủ đề (Topics), mini test (MiniTests); quản lý người dùng (Students); duyệt nội dung (Content Review); thống kê dashboard; import CSV; audit logs.

- **Creator Controller & Creator Service**

Xử lý logic cho biên tập viên: CRUD nội dung (từ vựng, câu hỏi, mini test), gửi duyệt nội dung (submit-review), xem thống kê (content-summary, analytics). Nội dung tạo ở trạng thái Draft, gửi admin duyệt qua PendingReview.

- **GamificationService**

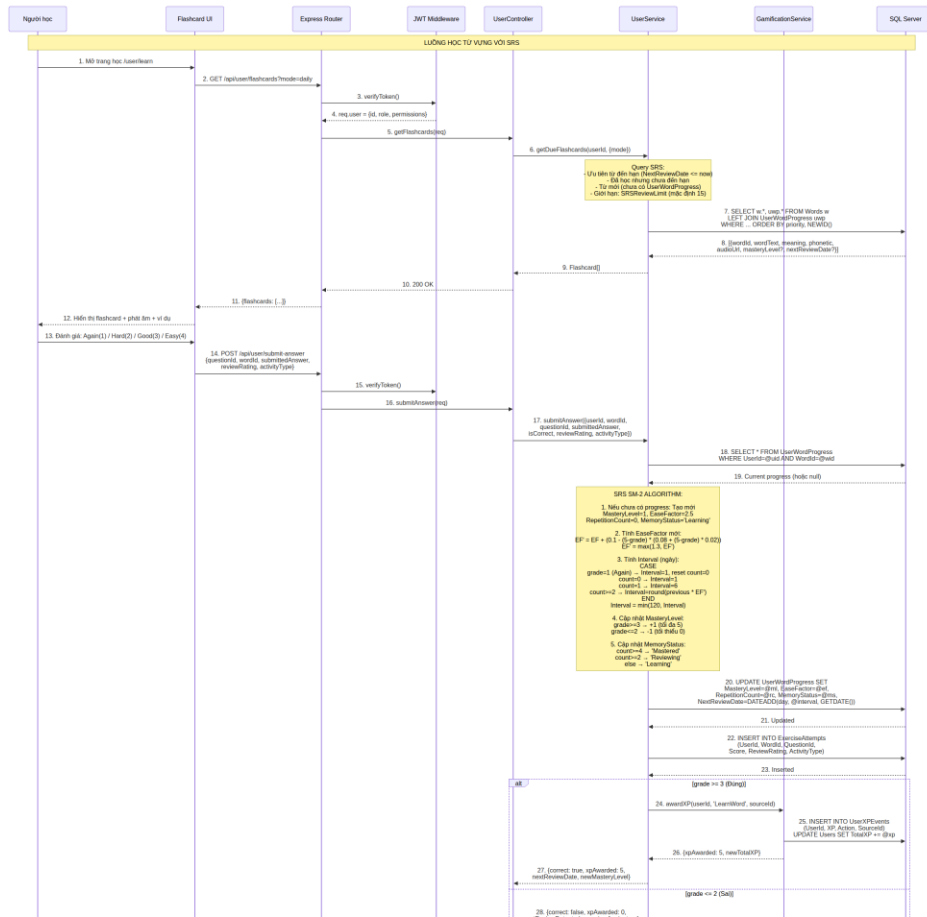
Dịch vụ gamification xử lý XP, Level, Streak, Achievement. Định nghĩa XP_REWARDS cho các hành động: LearnWord (10 XP), PracticeComplete (50 XP), MiniTestComplete (100 XP), DailyLogin (20 XP). Cung cấp getLevelState(totalXP) để tính cấp độ hiện tại ($\text{Level} = \text{floor}(\text{totalXP}/100) + 1$) và tiến độ XP trong cấp độ. Hàm awardXP(userId, action, sourceId) ghi nhận sự kiện XP. Theo dõi streak (số ngày học liên tiếp) và achievements (huy hiệu).

- **LearningPathService**

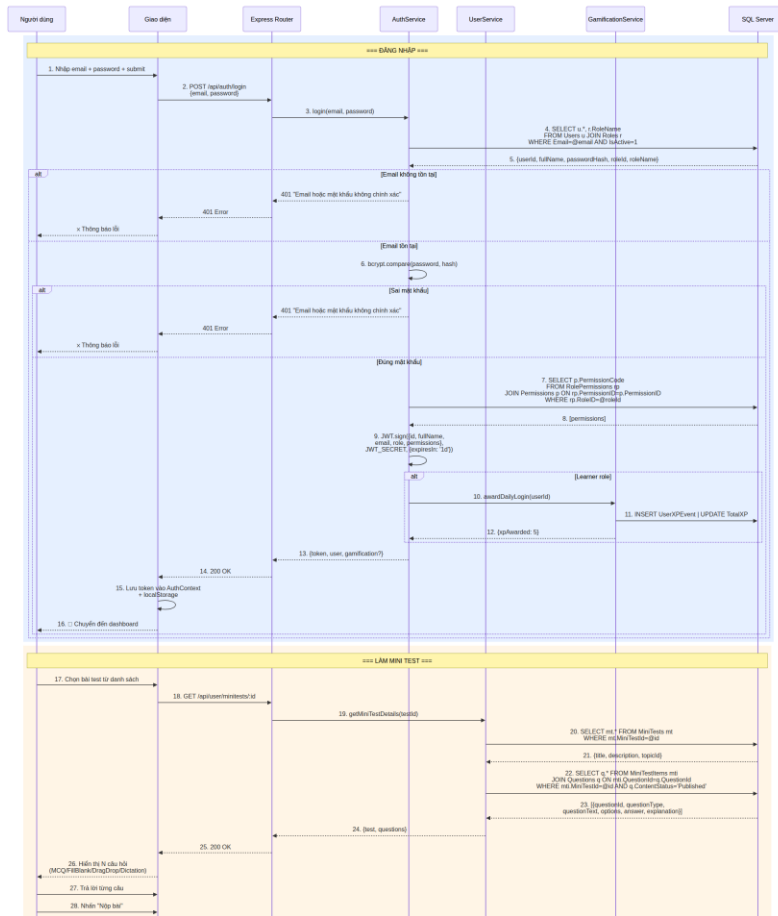
Dịch vụ quản lý lộ trình học tập (Learning Path). Tự động tạo schema với 2 bảng: dbo.LearningPathLevels (LevelId, Name, DisplayOrder) và dbo.LearningPathTopics (liên kết Level với Topics). Định nghĩa 4 cấp độ TOEIC: TOEIC 300, TOEIC 500, TOEIC 700, TOEIC 900. Mỗi cấp độ chứa nhiều chủ đề (Topics) tương ứng. getLearningPath(userId, levelId) trả về lộ trình cá nhân hóa với tiến độ học tập của người dùng tại từng chủ đề.

- **SQL Server Database (ToeicVocabularyPlatform)**

Cơ sở dữ liệu SQL Server (ToeicVocabularyPlatform) sử dụng mssql/msnodesqlv8 driver, hỗ trợ Windows Authentication và SQL Authentication. Tất cả queries được tham số hóa (parameterized queries) để chống SQL injection.



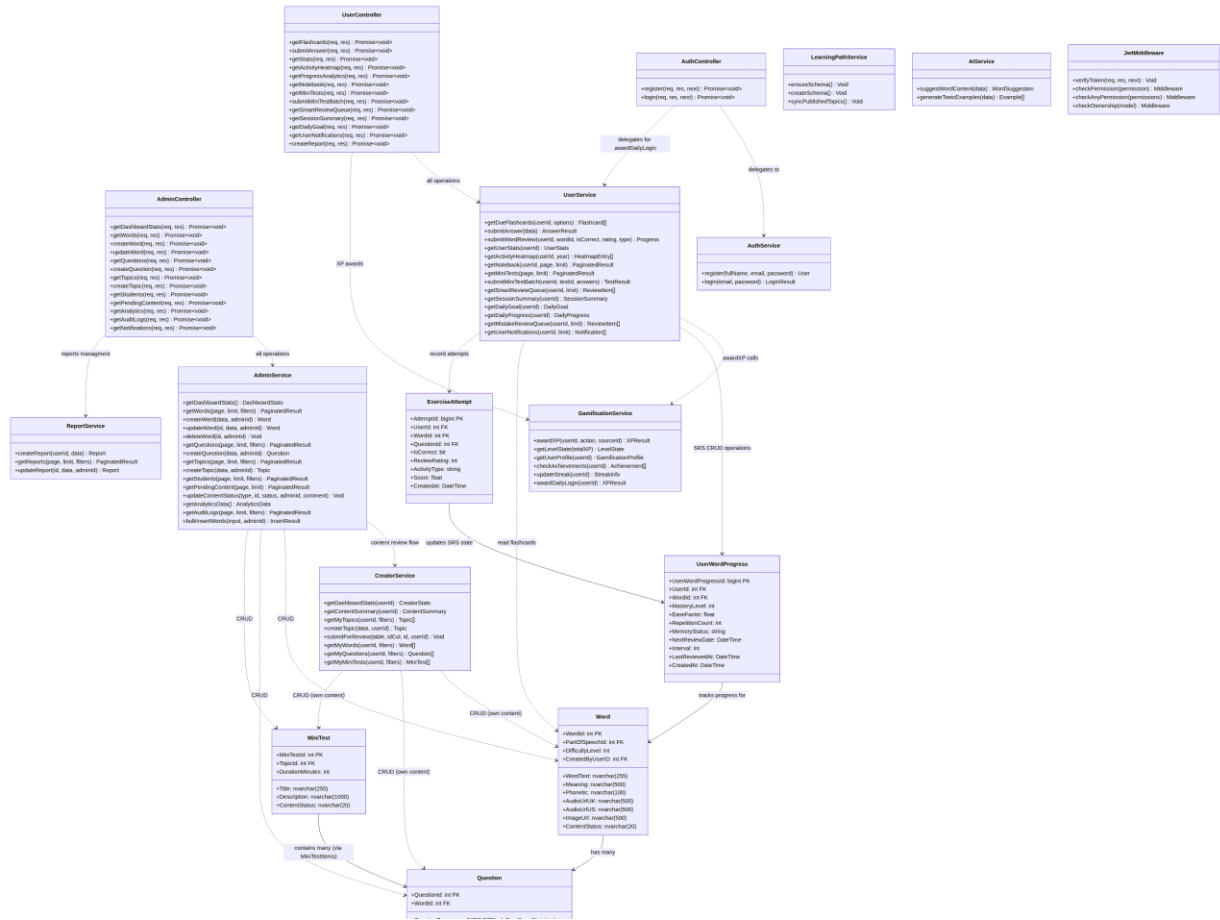
Hình 6: Sequence Diagram cho chức năng học từ vựng (SubmitAnswer)



Hình 7: Sequence Diagram cho chức năng đăng nhập và tạo Mini Test

4. THIẾT KẾ LỚP

4.1. Class Diagram



Hình 8: Biểu đồ lớp cho hệ thống VocaBoost

AuthController

Loại	Tên hàm / module	Mô tả chi tiết và giả mã
Nhóm	Nhóm xác thực và phân quyền	Nhóm hàm bảo vệ toàn bộ REST API: đăng ký, đăng nhập, sinh JWT, kiểm tra quyền và kiểm tra dữ liệu đầu vào.
Vai trò	Guest, Learner, Creator, Admin	Các hàm bên dưới được chọn vì trực tiếp thể hiện nghiệp vụ cốt lõi, quyền hạn người dùng hoặc luồng xử lý chính của hệ thống VocaBoost. Ký hiệu: S = Service, M = Middleware.
Loại	Signature	Chi tiết thực hiện
S	AuthService.register(fullName, email, password) <i>backend/src/services/auth.service.js</i>	Vai trò: Tạo tài khoản mới cho người học và gán vai trò mặc định Learner. Giả mã: FUNCTION register(fullName, email, password) Kết nối CSDL Tìm user có Email = email IF email đã tồn tại THEN Báo lỗi "Email already registered" END IF Hash password bằng bcrypt Lấy RoleID của vai trò Learner INSERT Users(fullName, email, passwordHash,

		<pre>roleId, isActive) Trả về thông tin user mới END FUNCTION</pre> Ghi chú: Hàm này thuộc luồng Guest -> Learner, không trả passwordHash ra client.
S	AuthService.login(email, password) <i>backend/src/services/auth.service.js</i>	Vai trò: Xác thực đăng nhập và cấp JWT token để frontend gọi các API được bảo vệ. Giải mã: <pre>FUNCTION login(email, password) Tìm user đang active theo email IF không tìm thấy THEN báo lỗi 401 So sánh password với PasswordHash bằng bcrypt IF mật khẩu sai THEN báo lỗi 401 Lấy danh sách permission theo RoleID Tạo JWT chứa id, email, role, permissions IF user là Learner THEN Thưởng daily login XP nếu đủ điều kiện END IF Trả về token và thông tin user END FUNCTION</pre> Ghi chú: Token là cơ sở cho middleware verifyToken và phân quyền động.
M	verifyToken(req, res, next) <i>backend/src/middlewares/auth.js</i>	Vai trò: Kiểm tra request có JWT hợp lệ trước khi cho truy cập API nội bộ. Giải mã: <pre>FUNCTION verifyToken(req, res, next) Đọc Authorization header IF header không có Bearer token THEN Trả về HTTP 401 END IF Giải mã token bằng JWT_SECRET IF token sai hoặc hết hạn THEN Trả về HTTP 401 END IF Gán thông tin user vào req.user Chuyển sang middleware/controller tiếp theo END FUNCTION</pre>
M	checkPermission(permission) <i>backend/src/middlewares/auth.js</i>	Vai trò: Bảo vệ endpoint chỉ cho user có đúng quyền cụ thể, ví dụ MANAGE_WORDS hoặc VIEW_DASHBOARD. Giải mã: <pre>FUNCTION checkPermission(permission) RETURN middleware(req, res, next) permissions = req.user.permissions IF permissions không chứa permission THEN Trả về HTTP 403 END IF Cho phép đi tiếp END RETURN END FUNCTION</pre>
M	checkAnyPermission(permissions) <i>backend/src/middlewares/auth.js</i>	Vai trò: Cho phép một endpoint được truy cập bởi nhiều nhóm quyền khác nhau. Giải mã: <pre>FUNCTION checkAnyPermission(permissions) RETURN middleware(req, res, next) userPermissions = req.user.permissions hasPermission = tồn tại quyền trong permissions mà user có IF hasPermission = false THEN Trả về HTTP 403 END IF Cho phép đi tiếp END RETURN END FUNCTION</pre>
M	validate(schema) <i>backend/src/middlewares/validate.js</i>	Vai trò: Kiểm tra dữ liệu request body/query/params trước khi vào controller. Giải mã: <pre>FUNCTION validate(schema) RETURN middleware(req, res, next)</pre>

		<pre> Chạy schema.validate(req.body) IF dữ liệu không hợp lệ THEN Trả về HTTP 400 kèm danh sách lỗi END IF Gán dữ liệu đã chuẩn hóa vào req.body Cho phép đi tiếp END RETURN END FUNCTION </pre>
--	--	--

LearningPathService

Loại	Tên hàm / module	Mô tả chi tiết và giả mã
Nhóm	Nhóm người học: chủ đề, lộ trình, SRS, mini-test và tiến độ	Nhóm hàm quan trọng nhất của dự án, thể hiện quá trình học từ vựng TOEIC, ôn tập lặp lại ngắt quãng và theo dõi tiến độ cá nhân.
Vai trò	Learner	Các hàm bên dưới được chọn vì trực tiếp thể hiện nghiệp vụ cốt lõi, quyền hạn người dùng hoặc luồng xử lý chính của hệ thống VocaBoost. Ký hiệu: S = Service, M = Middleware.
Loại	Signature	Chi tiết thực hiện
S	CategoriesService.getTopics(userId) <i>backend/src/services/categories.service.js</i>	Vai trò: Hiển thị danh sách chủ đề kèm thống kê tiến độ học của từng người dùng. Giả mã: <pre> FUNCTION getTopics(userId) Kết nối CSDL Lấy danh sách Topics đang Published JOIN WordTopics và Words để đếm wordCount IF có userId THEN JOIN UserWordProgress theo userId Tính learnedCount, masteredCount, dueCount Tính averageMastery và progressPercent ELSE Trả về thống kê cơ bản của topic END IF Sắp xếp theo displayOrder/topicName Trả về danh sách topics END FUNCTION </pre>
S	LearningPathService.getRoadmap(userId) <i>backend/src/services/learning-path.service.js</i>	Vai trò: Tạo lộ trình học theo các level TOEIC và trạng thái hoàn thành của learner. Giả mã: <pre> FUNCTION getRoadmap(userId) Đảm bảo schema LearningPath đã tồn tại Đồng bộ các topic Published vào roadmap Lấy danh sách level và topic thuộc từng level Lấy tiến độ học của user nếu có userId Gọi buildRoadmap để gom nhóm dữ liệu Trả về roadmap cho frontend hiển thị END FUNCTION </pre>
S	UserService.getDueFlashcards(userId, options) <i>backend/src/services/user.service.js</i>	Vai trò: Lấy danh sách flashcard cần học/ôn dựa trên lịch SRS và bộ lọc topic. Giả mã: <pre> FUNCTION getDueFlashcards(userId, options) Đọc topicId và mode từ options Lấy giới hạn ôn tập của user Query Words Published LEFT JOIN UserWordProgress theo userId IF mode = "new" THEN lấy từ chưa học IF mode = "daily" THEN lấy từ có NextReviewDate <= hiện tại Ưu tiên từ đến hạn, từ yếu, sau đó từ mới Trả về danh sách flashcards END FUNCTION </pre> Ghi chú: Đây là nơi thể hiện rõ kỹ thuật Spaced Repetition ở bước chọn từ cần ôn.

S	UserService.submitAnswer(payload) <i>backend/src/services/user.service.js</i>	Vai trò: Ghi nhận đáp án câu hỏi, cập nhật phản hồi ôn tập và cộng XP nếu là hoạt động học từ. Giải mã: <pre> FUNCTION submitAnswer(payload) Nhận userId, questionId, submittedAnswer, reviewRating Gọi stored procedure usp_SubmitQuestionAttempt Lấy WordID chuẩn từ kết quả câu hỏi IF không lấy được WordID THEN báo lỗi IF reviewRating thuộc Again/Hard/Good/Easy THEN Cập nhật EaseFactor và NextReviewDate Again -> ôn lại sau 10 phút Hard -> ôn lại sau 1 ngày Good/Easy -> giãn ngày theo MasteryLevel END IF IF activityType = LearnWord THEN cộng XP Trả về kết quả chấm, lịch ôn tiếp theo và XP END FUNCTION </pre>
S	UserService.submitWordReview(payload) <i>backend/src/services/user.service.js</i>	Vai trò: Cập nhật trạng thái ghi nhớ của một từ sau khi learner tự đánh giá flashcard. Giải mã: <pre> FUNCTION submitWordReview(payload) Nhận userId, wordId, isCorrect, reviewRating MERGE UserWordProgress theo userId + wordId IF đã có tiến độ THEN Tăng/giảm MasteryLevel theo isCorrect Cập nhật ConsecutiveCorrect/ConsecutiveWrong Tính NextReviewDate theo reviewRating Điều chỉnh EaseFactor trong khoảng cho phép Cập nhật MemoryStatus: Learning/Reviewing/Mastered/Lapsed ELSE Tạo bản ghi tiến độ ban đầu cho từ END IF IF activityType = LearnWord THEN cộng XP Trả về masteryLevel, memoryStatus, nextReviewDate END FUNCTION </pre> Ghi chú: Hàm này là lõi của SRS engine trong dự án.
S	UserService.getSmartReviewQueue(userId, limit) <i>backend/src/services/user.service.js</i>	Vai trò: Tạo hàng đợi ôn tập thông minh, ưu tiên từ sắp quên hoặc sai nhiều. Giải mã: <pre> FUNCTION getSmartReviewQueue(userId, limit) Chuẩn hóa limit trong khoảng 1..50 Lấy Words Published đã có UserWordProgress Chỉ lấy từ có NextReviewDate trong 7 ngày tới Tính priorityScore: quá hạn nhiều giờ -> điểm ưu tiên cao có ConsecutiveWrong -> nhân hệ số ưu tiên chưa đến hạn -> ưu tiên thấp hơn Sắp xếp theo priorityScore và masteryLevel Trả về danh sách từ cần ôn END FUNCTION </pre>
S	UserService.submitMiniTestBatch(userId, testId, answers) <i>backend/src/services/user.service.js</i>	Vai trò: Chấm toàn bộ mini-test, lưu lịch sử làm bài và cập nhật tiến độ từng từ. Giải mã: <pre> FUNCTION submitMiniTestBatch(userId, testId, answers) Bắt đầu transaction Lấy danh sách câu hỏi thuộc miniTest IF miniTest không có câu hỏi THEN báo lỗi IF số đáp án không khớp THEN báo lỗi FOR mỗi answer Kiểm tra question có thuộc miniTest không So sánh submittedAnswer với CorrectAnswer INSERT ExerciseAttempts MERGE UserWordProgress cho WordID tương ứng Cộng correctCount nếu đúng END FOR Commit transaction END FUNCTION </pre>

		END FOR INSERT MiniTestAttempts với điểm tổng Commit transaction Cộng XP hoàn thành mini-test Trả về score, correctCount và chi tiết từng câu END FUNCTION
S	UserService.getProgressAnalytics(userId) <i>backend/src/services/user.service.js</i>	Vai trò: Tổng hợp dữ liệu biểu đồ dashboard học tập của người học. Giải mã: FUNCTION getProgressAnalytics(userId) Lấy tổng số từ đã học và đã mastered Tính phân bố MasteryLevel Tính activity theo ngày/tuần/tháng Tính tiến độ theo topic/level Lấy accuracy từ ExerciseAttempts Gom dữ liệu thành analytics object Trả về dữ liệu cho Learning Dashboard END FUNCTION
S	UserService.getMistakeReviewQueue(userId, limit) <i>backend/src/services/user.service.js</i>	Vai trò: Lấy danh sách từ/câu hỏi learner hay sai để ôn lại có trọng tâm. Giải mã: FUNCTION getMistakeReviewQueue(userId, limit) Chuẩn hóa limit Query ExerciseAttempts của user Lọc các attempt sai GROUP BY WordID/QuestionID để đếm số lần sai JOIN Words và Questions để lấy nội dung ôn Sắp xếp theo số lỗi và thời điểm sai gần nhất Trả về danh sách lỗi cần ôn END FUNCTION
S	UserService.addNotebookEntry(userId, data) <i>backend/src/services/user.service.js</i>	Vai trò: Cho phép learner lưu từ vào sổ tay cá nhân kèm ghi chú riêng. Giải mã: FUNCTION addNotebookEntry(userId, data) Nhận wordId, note, isFavorite Kiểm tra word tồn tại Kiểm tra entry đã có trong notebook chưa IF đã tồn tại THEN cập nhật ghi chú/isFavorite ELSE INSERT UserVocabularyNotebook END IF Trả về entry mới nhất END FUNCTION

AdminService

Loại	Tên hàm / module	Mô tả chi tiết và giải mã
Nhóm	Nhóm quản trị và biên tập nội dung	Nhóm hàm phục vụ quản lý kho từ vựng TOEIC, câu hỏi, mini-test, import dữ liệu và phân quyền người dùng.
Vai trò	Admin, Content Creator	Các hàm bên dưới được chọn vì trực tiếp thể hiện nghiệp vụ cốt lõi, quyền hạn người dùng hoặc luồng xử lý chính của hệ thống VocaBoost. Ký hiệu: S = Service, M = Middleware.
Loại	Signature	Chi tiết thực hiện
S	AdminService.getDashboardStats() <i>backend/src/services/admin.service.js</i>	Vai trò: Tổng hợp số liệu cho trang Admin Dashboard. Giải mã: FUNCTION getDashboardStats() Đếm tổng Users, Words, Questions, Topics, MiniTests Đếm số nội dung PendingReview Đếm số learner đang hoạt động Tính accuracy/attempt trung bình nếu có dữ liệu Gom các chỉ số thành object thống kê

		Trả về dashboard stats END FUNCTION
S	AdminService.getTopics(page, limit, filters) <i>backend/src/services/admin.service.js</i>	Vai trò: Quản trị danh sách chủ đề, hỗ trợ tìm kiếm, lọc trạng thái và phân trang. Giải mã: <pre> FUNCTION getTopics(page, limit, filters) Chuẩn hóa page và limit Tạo điều kiện WHERE theo search/status/categoryId Query Topics kèm CategoryName Đếm wordCount và miniTestCount của từng topic Áp dụng ORDER BY và OFFSET/FETCH Query tổng số bản ghi Trả về { data, total, page, limit } END FUNCTION </pre>
S	AdminService.createWord(wordData, adminId) <i>backend/src/services/admin.service.js</i>	Vai trò: Admin thêm từ vựng TOEIC mới cùng topic và câu ví dụ. Giải mã: <pre> FUNCTION createWord(wordData, adminId) Kiểm tra ContentStatus hợp lệ Chuẩn hóa topicIds và examples Bắt đầu transaction INSERT Words(term, meaning, phonetic, partOfSpeechId, status) FOR mỗi topicId INSERT WordTopics(wordId, topicId) END FOR FOR mỗi example hợp lệ INSERT ExampleSentences(wordId, sentence, translation) END FOR Commit transaction Ghi AdminAuditLog CREATE_WORD Trả về id và thông tin từ vừa tạo END FUNCTION </pre>
S	AdminService.updateWord(wordId, wordData, adminId) <i>backend/src/services/admin.service.js</i>	Vai trò: Cập nhật từ vựng, topic liên kết và ví dụ minh họa. Giải mã: <pre> FUNCTION updateWord(wordId, wordData, adminId) Kiểm tra status hợp lệ Chuẩn hóa topicIds/examples nếu được gửi lên Bắt đầu transaction UPDATE Words theo wordId IF không có dòng bị cập nhật THEN rollback và trả false IF có topicIds THEN Xóa WordTopics cũ Thêm lại WordTopics mới END IF IF có examples THEN Xóa ExampleSentences cũ Thêm lại examples mới END IF Commit transaction Ghi audit log UPDATE_WORD Trả về true END FUNCTION </pre>
S	AdminService.previewWordImport(input) <i>backend/src/services/admin.service.js</i>	Vai trò: Đọc trước file CSV/plain text để admin kiểm tra lỗi trước khi import thật. Giải mã: <pre> FUNCTION previewWordImport(input) Parse nội dung CSV/text thành các dòng dữ liệu FOR mỗi dòng Chuẩn hóa key: term, meaning, phonetic, topics, examples Kiểm tra trường bắt buộc Kiểm tra trùng từ hoặc dữ liệu sai định dạng Đưa vào danh sách validRows hoặc errors END FOR Trả về preview gồm tổng dòng, dòng hợp lệ, dòng lỗi END FUNCTION </pre>

S	AdminService.bulkInsertWords(input, adminId) <i>backend/src/services/admin.service.js</i>	Vai trò: Import hàng loạt từ vựng để xây dựng nhanh ngân hàng TOEIC. Giải mã: <pre> FUNCTION bulkInsertWords(input, adminId) Parse dữ liệu import Bắt đầu transaction FOR mỗi word hợp lệ INSERT Words Tạo Topic nếu cần hoặc lấy topic hiện có INSERT WordTopics INSERT ExampleSentences nếu có END FOR Nếu có lỗi nghiêm trọng THEN rollback Commit transaction Ghi audit log BULK_IMPORT_WORDS Trả về số dòng thành công/thất bại END FUNCTION </pre>
S	AdminService.createMiniTest(testData, adminId) <i>backend/src/services/admin.service.js</i>	Vai trò: Admin tạo bài kiểm tra nhanh theo topic và danh sách câu hỏi. Giải mã: <pre> FUNCTION createMiniTest(testData, adminId) Kiểm tra topic và danh sách questionIds Bắt đầu transaction INSERT MiniTests(title, description, topicId, status) FOR mỗi questionId INSERT MiniTestItems(miniTestId, questionId, displayOrder) END FOR Commit transaction Ghi audit log CREATE_MINITEST Trả về miniTest vừa tạo END FUNCTION </pre>
S	AdminService.updateUserRole(userId, roleName) <i>backend/src/services/admin.service.js</i>	Vai trò: Admin đổi vai trò người dùng giữa Learner, Creator hoặc Admin. Giải mã: <pre> FUNCTION updateUserRole(userId, roleName) Kiểm tra roleName hợp lệ Tìm RoleID theo roleName IF role không tồn tại THEN báo lỗi UPDATE Users SET RoleID = roleId IF không có user bị cập nhật THEN báo lỗi Trả về user với role mới END FUNCTION </pre>
S	CreatorService.createWord(data, userId) <i>backend/src/services/creator.service.js</i>	Vai trò: Content Creator tạo từ vựng ở trạng thái Draft để gửi duyệt. Giải mã: <pre> FUNCTION createWord(data, userId) Chuẩn hóa topicIds Bắt đầu transaction Kiểm tra PartOfSpeech tồn tại Kiểm tra từ chưa bị trùng theo Term + PartOfSpeech FOR mỗi topicId Kiểm tra creator được dùng topic đó END FOR INSERT Words với ContentStatus = Draft INSERT WordTopics và ExampleSentences Commit transaction Trả về id từ vừa tạo END FUNCTION </pre>
S	CreatorService.createQuestion(data, userId) <i>backend/src/services/creator.service.js</i>	Vai trò: Content Creator tạo câu hỏi luyện tập gắn với từ vựng. Giải mã: <pre> FUNCTION createQuestion(data, userId) Kiểm tra WordID tồn tại và creator được sử dụng Kiểm tra dạng câu hỏi, đáp án đúng và options Bắt đầu transaction INSERT Questions với ContentStatus = Draft Nếu có options thì INSERT QuestionOptions Commit transaction </pre>

		Trả về question vừa tạo END FUNCTION
S	CreatorService.createMiniTest(data, userId) <i>backend/src/services/creator.service.js</i>	Vai trò: Content Creator tạo mini-test trước khi gửi admin xét duyệt. Giải mã: FUNCTION createMiniTest(data, userId) Kiểm tra topic và questionIds hợp lệ Bắt đầu transaction INSERT MiniTests với ContentStatus = Draft FOR mỗi questionId INSERT MiniTestItems END FOR Commit transaction Trả về mini-test mới END FUNCTION
S	CreatorService.submitForReview(tableName, idColumn, id, userId) <i>backend/src/services/creator.service.js</i>	Vai trò: Chuyển nội dung của creator từ Draft/Rejected sang PendingReview. Giải mã: FUNCTION submitForReview(tableName, idColumn, id, userId) Kiểm tra entity thuộc user hiện tại IF ContentStatus không phải Draft hoặc Rejected THEN báo lỗi UPDATE entity SET ContentStatus = PendingReview Ghi thời điểm cập nhật Trả về nội dung đã gửi duyệt END FUNCTION

UserService

Loại	Tên hàm / module	Mô tả chi tiết và giải mã
Nhóm	Nhóm duyệt nội dung, AI, gamification và thông báo	Nhóm hàm hỗ trợ vận hành hệ thống: duyệt nội dung, sinh gọi ý AI, cộng điểm XP, mở khóa thành tích và gửi thông báo.
Vai trò	Admin, Creator, Learner	Các hàm bên dưới được chọn vì trực tiếp thể hiện nghiệp vụ cốt lõi, quyền hạn người dùng hoặc luồng xử lý chính của hệ thống VocaBoost. Ký hiệu: S = Service, M = Middleware.
Loại	Signature	Chi tiết thực hiện
S	ReviewService.getPendingContent() <i>backend/src/services/review.service.js</i>	Vai trò: Admin lấy toàn bộ nội dung đang chờ duyệt từ nhiều bảng. Giải mã: FUNCTION getPendingContent() Query Topics có ContentStatus = PendingReview UNION với Words PendingReview UNION với Questions PendingReview UNION với MiniTests PendingReview JOIN Users để lấy creatorName Sắp xếp theo createdAt tăng dần Trả về danh sách nội dung chờ duyệt END FUNCTION
S	ReviewService.approve(entityType, entityId, adminId) <i>backend/src/services/review.service.js</i>	Vai trò: Admin phê duyệt nội dung, chuyển từ PendingReview sang Published. Giải mã: FUNCTION approve(entityType, entityId, adminId) Resolve entityType thành table và khóa chính Bắt đầu transaction UPDATE entity SET ContentStatus = Published Cập nhật ReviewedByUserID, ReviewedAt, PublishedAt nếu có IF entity là MiniTest THEN set IsPublished = 1 INSERT ContentReviewLogs với NewStatus = Published

		Commit transaction Trả về true nếu duyệt thành công END FUNCTION
S	ReviewService.reject(entityType, entityId, adminId, reason) <i>backend/src/services/review.service.js</i>	Vai trò: Admin từ chối nội dung và lưu lý do để creator chỉnh sửa. Giải mã: FUNCTION reject(entityType, entityId, adminId, reason) Resolve entityType thành table và khóa chính Bắt đầu transaction UPDATE entity SET ContentStatus = Rejected Cập nhật ReviewedByUserID và ReviewedAt INSERT ContentReviewLogs với comment = reason Commit transaction Trả về true nếu từ chối thành công END FUNCTION
S	AiService.suggestWordContent(payload) <i>backend/src/services/ai.service.js</i>	Vai trò: Gợi ý nội dung từ vựng TOEIC cho creator/admin: nghĩa, ví dụ, giải thích. Giải mã: FUNCTION suggestWordContent(payload) Nhận term, meaning, partOfSpeech, exampleCount Làm sạch dữ liệu đầu vào Tạo prompt yêu cầu sinh nội dung theo ngữ cảnh TOEIC Gọi AI provider với timeout Trích xuất response text Parse JSON từ response Chuẩn hóa danh sách ví dụ Trả về nội dung gợi ý cho frontend END FUNCTION
S	GamificationService.awardXP(userId, event) <i>backend/src/services/gamification.service.js</i>	Vai trò: Cộng điểm kinh nghiệm cho learner sau các hoạt động học tập. Giải mã: FUNCTION awardXP(userId, event) Đảm bảo schema gamification tồn tại Xác định số XP theo eventType IF sourceKey đã được thưởng trước đó THEN Không cộng trùng XP END IF INSERT UserXPEvents Cập nhật TotalXP của Users Tính level mới từ tổng XP Kiểm tra achievement mới Trả về xpGained, totalXP, levelState END FUNCTION
S	GamificationService.checkAchievements(userId) <i>backend/src/services/gamification.service.js</i>	Vai trò: Kiểm tra điều kiện mở khóa huy hiệu/thành tích của learner. Giải mã: FUNCTION checkAchievements(userId) Lấy metrics học tập của user Lấy danh sách achievements hiện có FOR mỗi achievement Kiểm tra điều kiện: XP, streak, số từ mastered, mini-test IF đạt điều kiện và chưa nhận THEN INSERT UserAchievements Đưa vào danh sách newlyUnlocked END IF END FOR Trả về achievement mới mở khóa END FUNCTION
S	AdminService.sendAnnouncement(data, adminId) <i>backend/src/services/admin.service.js</i>	Vai trò: Admin gửi thông báo hệ thống cho learner/creator hoặc toàn bộ người dùng. Giải mã: FUNCTION sendAnnouncement(data, adminId) Kiểm tra title, message, targetRole/targetUser Xác định danh sách người nhận FOR mỗi user nhận thông báo INSERT Notifications(userId, title, message, type) END FOR

		Ghi audit log SEND_ANNOUNCEMENT Trả về số thông báo đã tạo END FUNCTION
S	AdminService.createDailyReminders(adminId) <i>backend/src/services/admin.service.js</i>	Vai trò: Tạo nhắc nhở học tập hằng ngày cho người học còn từ đến hạn ôn. Giải mã: FUNCTION createDailyReminders(adminId) Tìm learner đang active Với mỗi learner, đếm từ có NextReviewDate <= hiện tại IF learner có từ cần ôn THEN INSERT Notification nhắc học hôm nay END IF Ghi audit log CREATE_DAILY_REMINDERS Trả về số reminder đã tạo END FUNCTION

AdminController

Name		Description
Properties		
	LevelId: int (PK)	ID tự tăng, khóa chính của cấp độ. <i>Giải thích: 4 cấp độ TOEIC được seed sẵn: 300, 500, 700, 900. Mỗi cấp độ chứa nhiều chủ đề học tập thông qua bảng LearningPathTopics.</i>
	Name: nvarchar(100)	Tên cấp độ: "TOEIC 300", "TOEIC 500", "TOEIC 700", "TOEIC 900".
DisplayOrder: int - Thứ tự hiển thị trong lộ trình học tập (1-4).		
Phức	Method	getTopics(): LearningPathTopic[] - lấy danh sách chủ đề thuộc cấp độ này. Pseudo-code: 1. SELECT t.* FROM LearningPathTopics lpt JOIN Topics t ON lpt.TopicId = t.TopicId WHERE lpt.LevelId = @levelId AND t.ContentStatus = 'Published' 2. Với mỗi topic, đếm số words và tỷ lệ hoàn thành của user (nếu có userId) 3. Return: [{ topicId, name, wordCount, completedPercentage? }]

Word

Name		Description
Properties		
	WordId: int (PK)	ID tự tăng, khóa chính của từ vựng. <i>Bảng Words là bảng trung tâm - lưu ~720+ từ vựng TOEIC với đầy đủ multimedia (audio Anh-Anh, Anh-Mỹ, hình ảnh, phiên âm).</i>
	WordText: nvarchar(255)	Từ vựng tiếng Anh (ví dụ: "abandon", "negotiate"). <i>Các từ vựng được phân loại theo 4 cấp độ TOEIC (300-500-700-900). VD: 'abandon' (cấp 300), 'negotiate' (cấp 500), 'consolidate' (cấp 700).</i>
	Meaning: nvarchar(500)	Nghĩa tiếng Việt của từ vựng.
	PartOfSpeechId: int (FK)	Khóa ngoại -> PartOfSpeeches. Loại từ: noun, verb, adjective, adverb...
Method		
	getTopics(): Topic[]	Lấy danh sách chủ đề chứa từ vựng (qua bảng trung gian WordTopics). Pseudo-code: 1. SELECT t.* FROM WordTopics wt JOIN Topics t ON wt.TopicId = t.TopicId WHERE wt.WordId = @wordId 2. Mỗi từ có thể thuộc nhiều chủ đề (quan hệ N-N qua WordTopics) 3. Return: [{ topicId, name, categoryName }]
	getQuestions(): Question[]	Lấy danh sách câu hỏi luyện tập liên quan đến từ vựng. Pseudo-code: 1. SELECT * FROM Questions WHERE WordId = @wordId

		AND ContentStatus = 'Published' 2. Các loại câu hỏi: MCQ (trắc nghiệm), FillBlank (điền từ), DragDrop (sắp xếp), Dictation (nghe viết) 3. Mỗi từ thường có 2-5 câu hỏi với các dạng khác nhau
--	--	--

UserService

Name		Description
Properties		
	UserWordProgressId: bigint (PK)	ID tự tăng, khóa chính. <i>Bảng quan trọng nhất cho SRS Engine. Mỗi record tương ứng với 1 từ vựng của 1 người dùng. Lưu trữ toàn bộ trạng thái học tập SRS.</i>
	UserId: int (FK)	Khóa ngoại -> Users. <i>FK → Users. Mỗi user có ~10-500+ UserWordProgress records tùy vào tiến độ học.</i>
WordId: int (FK) - Khóa ngoại -> Words.		
	MasteryLevel: int	Mức độ thành thạo (0-5, mặc định 1). Tăng khi đúng. <i>Các mức: 0=Chưa học, 1=Mới bắt đầu, 2=Đang học, 3=Khá, 4=Tốt, 5=Thành thạo.</i> <i>Tăng khi đúng (grade >= 3), giảm khi sai (grade <= 2).</i>
	EaseFactor: float	Hệ số dễ SRS. Mặc định 2.5. Điều chỉnh theo rating. <i>Công thức SM-2: $EF' = EF + (0.1 - (5 - grade) * (0.08 + (5 - grade) * 0.02))$</i> <i>EF không xuống dưới 1.3. Grade = Again(1), Hard(2), Good(3), Easy(4).</i> <i>EF càng cao → khoảng cách Interval càng dài → ôn tập ít hơn.</i>
	RepetitionCount: int	Số lần lặp lại thành công liên tiếp. <i>Số lần đánh giá 'Good' hoặc 'Easy' liên tiếp. Reset về 0 khi đánh giá 'Again'. Dùng để tính Interval: $0 \rightarrow 1, 1 \rightarrow 6, \geq 2 \rightarrow previous * EF$.</i>
	MemoryStatus: nvarchar(20)	Trạng thái ghi nhớ: "Learning", "Reviewing", "Mastered".

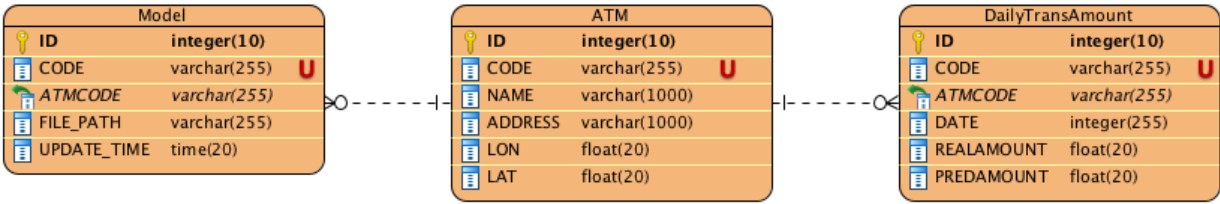
		Learning: Đang học lần đầu (cần ôn tập thường xuyên). Reviewing: Đã học qua, đang trong giai đoạn ôn tập (Interval từ 1-30 ngày). Mastered: Đã thành thạo (đạt 4+ lần đúng liên tiếp, Interval $\geq$ 60 ngày). Cơ chế chuyển trạng thái: Learning $\rightarrow$ Reviewing: sau 2 lần đúng liên tiếp Reviewing $\rightarrow$ Mastered: sau 4 lần đúng liên tiếp Bất kỳ $\rightarrow$ Learning: khi đánh giá 'Again' (sai)
--	--	--

UserWordProgress (SRS Engine)

Name		Description
Properties		
	MiniTestId: int (PK)	ID tự tăng, khóa chính. <i>Bảng MiniTests quản lý các bài kiểm tra nhanh do Admin/Creator tạo. Mỗi bài test có N câu hỏi (N=5-20) được liên kết qua bảng MiniTestItems.</i>
	Title: nvarchar(255)	Tiêu đề bài MiniTest.
	Description: nvarchar(1000)	Mô tả nội dung bài kiểm tra.
	TopicId: int (FK)	Khóa ngoại $\rightarrow$ Topics. Chủ đề của bài kiểm tra. <i>FK $\rightarrow$ Topics. Mỗi MiniTest thuộc một chủ đề cụ thể, giúp người học kiểm tra kiến thức theo từng chủ đề TOEIC.</i>
Method		
	getQuestions(): Question[]	Lấy danh sách câu hỏi trong bài kiểm tra (qua bảng MiniTestItems). Pseudo-code: <pre>1. SELECT q.* FROM MiniTestItems mti JOIN Questions q ON mti.QuestionId = q.QuestionId</pre>

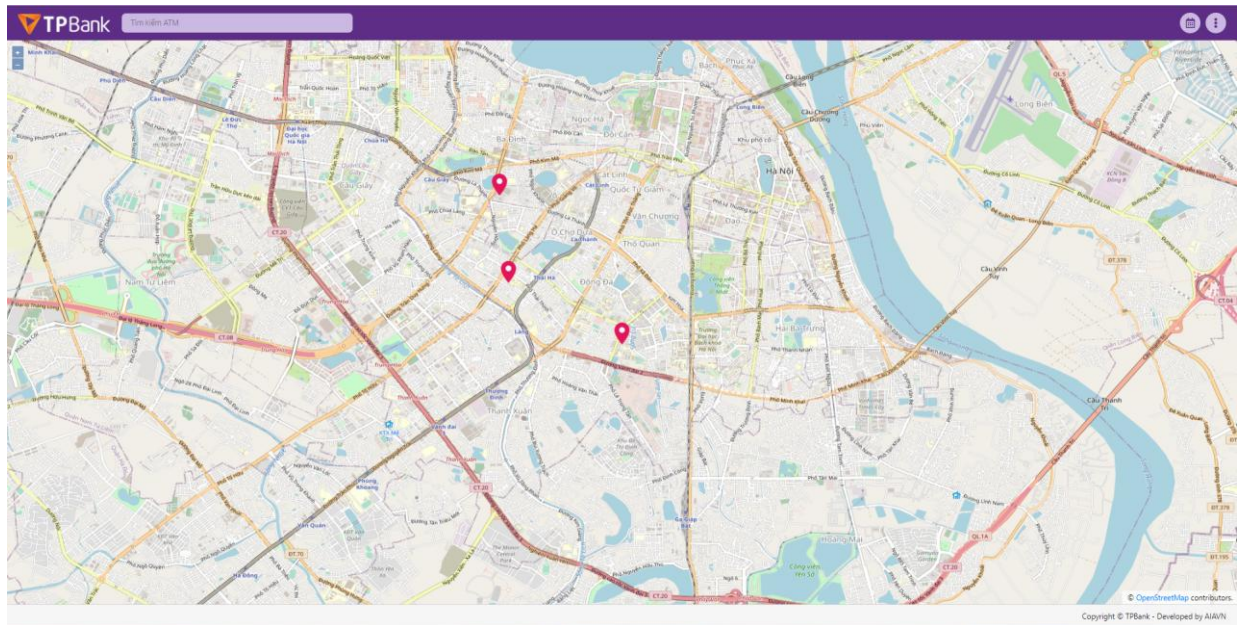
		WHERE mti.MiniTestId = @miniTestId AND q.ContentStatus = 'Published' 2. Trộn ngẫu nhiên thứ tự câu hỏi 3. Với mỗi câu hỏi, kiểm tra user đã làm chưa (nếu có userId) 4. Return: [{ questionId, questionType, questionText, options, correctAnswer?, explanation, userAttempted? }] Loại câu hỏi hỗ trợ: - MCQ: Multiple Choice (4 options) - FillBlank: Điền từ vào chỗ trống - DragDrop: Sắp xếp từ thành câu hoàn chỉnh - Dictation: Nghe và viết lại
--	--	--

5. THIẾT KẾ DỮ LIỆU



6. THIẾT KẾ GIAO DIỆN NGƯỜI DÙNG

6.1. Trang Dashboard (User) & Landing Page



6.2. Popup thông tin từ vừng



6.3. Giao diện học từ vựng Flashcard với thẻ SRS và 4 nút đánh giá (Again/Hard/Good/Easy)

ATM 001 (2 Chùa Bộc, Quang Trung, Đống Đa, Hà Nội, Việt Nam)



6.4. Giao diện Mini Test tổng hợp các câu hỏi trắc nghiệm từ vựng

Dự báo nhu cầu rút tiền

X

ATM CO...	Tên ATM	Địa chỉ	Dự báo (đơn vị: triệu đồng)							
				06/01/2021	07/01/2021	08/01/2021	09/01/2021	10/01/2021	11/01/2021	12/01/2021
ATM001	ATM 001	2 Chùa Bộc, Quang Trung, Đống Đa, Hà Nội...	Nhu cầu rút tiền Gợi ý tiếp quỹ	810 2340	710 2310	820 2450	780 2460	850 2490	830 -	810 -
ATM002	ATM 002	22 Láng Hạ, Đống Đa, Hà Nội, Việt Nam	Nhu cầu rút tiền Gợi ý tiếp quỹ	810 2380	750 2340	820 2440	770 2450	850 2480	830 -	800 -
ATM003	ATM 003	29 Nguyễn Chí Thanh, Ngọc Khánh, Đống D...	Nhu cầu rút tiền Gợi ý tiếp quỹ	810 2370	740 2340	820 2450	780 2460	850 2490	830 -	810 -

< > 1 >

1 of 1 pages (3 items)

6.5. Dialog danh sách từ vựng

Danh sách cây ATM

+

THÊM MỚI

ATM Code	Tên cây ATM	Địa chỉ	Kinh độ	Vĩ độ	
ATM001	ATM 001	2 Chùa Bộc, Quang Trung, Đống Đa, Hà Nội, Việt Nam	105.831029	21.006853	
ATM002	ATM 002	22 Láng Hạ, Đống Đa, Hà Nội, Việt Nam	105.814167	21.016007	
ATM003	ATM 003	29 Nguyễn Chí Thanh, Ngọc Khánh, Đống Đa, Hà Nội, Việt Nam	105.812826	21.028978	

1 of 1 pages (3 items)

6.6. Trang Quản trị (Admin Dashboard & CRUD)

Cấu hình hệ thống

×

Hệ số an toàn (VNĐ)

50000000

▼▲

Thời hạn tiếp quỹ

3 Ngày

▼▲

Thời điểm thực hiện cập nhật dữ liệu hàng ngày

23:00

🕒

LƯU LẠI

(Số trang sẽ tự động cập nhật khi mở file trong Microsoft Word và nhấn Ctrl+A → F9)